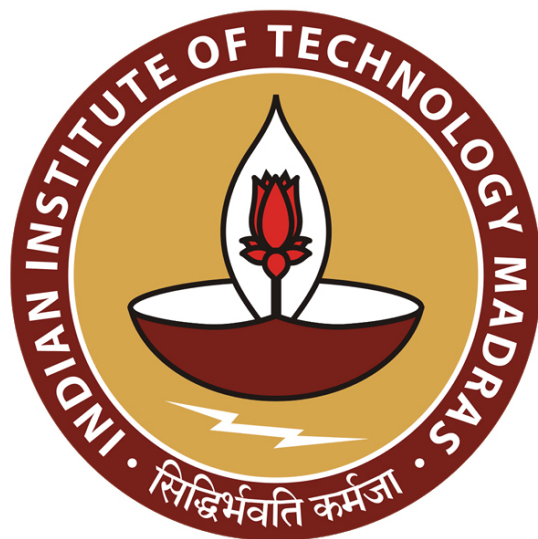


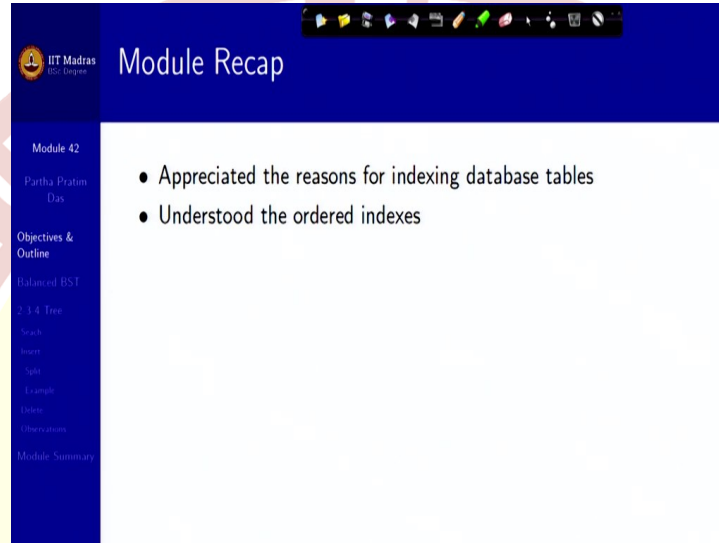


IIT Madras
ONLINE DEGREE

Database Management Systems
Professor Partha Pratim Das
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur
Indexing and Hashing/2: Indexing/2

Welcome to Module 42 of Database Management Systems course, in IIT Madras online B.Sc. program.

(Refer Slide Time: 00:26)

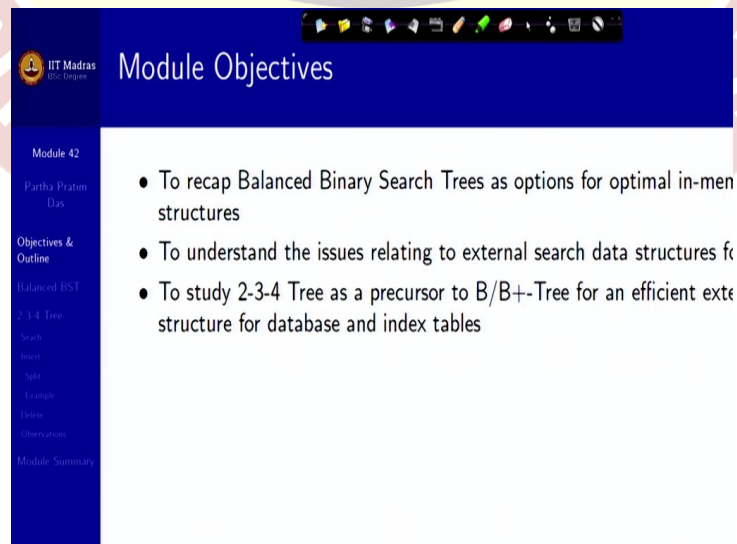


Module Recap

- Appreciated the reasons for indexing database tables
- Understood the ordered indexes

In the last module, we have introduced the concept of indexing and the need for that and we understood the fundamentals of ordered indices, primary index, secondary index, dense index, sparse index, multi-level index and all that.

(Refer Slide Time: 00:44)



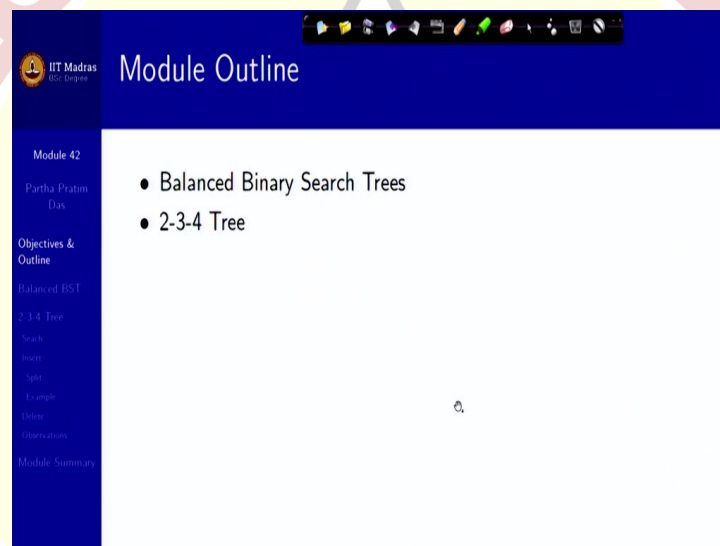
Module Objectives

- To recap Balanced Binary Search Trees as options for optimal in-memory structures
- To understand the issues relating to external search data structures for
- To study 2-3-4 Tree as a precursor to B/B+-Tree for an efficient external structure for database and index tables

Now, before we move further, we would again like to step back a little because the next that we want to understand is to be able to implement these indices and the actual index sequential access method, what kind of external structure in the disk we have to create, which happens to be B tree or B+ tree, which will be efficient as an internal structure catering to the basic requirement of my database relation data as well as my index tables.

So, in this module, I again step back a little I developed this concept from the in-memory data structure that we have seen extend them to a point, where they can be directly extended or actually kind of replicated in an external disk mechanism. So, that is what we will discuss today.

(Refer Slide Time: 01:54)



We will talk a little bit more about balanced binary search tree and specifically 2-3-4 trees.

(Refer Slide Time: 02:02)

Search Data Structures

- How to search a key in a list of n data items?
 - Linear Search: $O(n)$: Find 28 $\Rightarrow$ 16 comparisons
 - Unordered items in an array – search sequentially
 - Unordered / Ordered items in a list – search sequentially
 - Binary Search: $O(\lg n)$: Find 28 $\Rightarrow$ 4 comparisons – 25, 36, 30, 28
 - Ordered items in an array – search by divide-and-conquer
 - Binary Search Tree – recursively on left / right

```
graph TD; 25 --> 20; 25 --> 36; 20 --> 10; 20 --> 22; 36 --> 30; 36 --> 40
```

Now, we have talked about binary search trees, you know this pretty well linear search all these you know, order n , binary search order $\log n$ if it is sorted and we saw that this can be actually mimic in a nonlinear data structure, binary search tree where the search could be really made efficient in terms of going through the comparison over this binary tree and once the search is efficient, then the insertion or deletion can also be done efficiently by manipulating a few pointers. So, that is what we have learnt at the fundamental level.

(Refer Slide Time: 02:46)

Search Data Structures (2)

- Worst case time (n data items in the data structure):

Data Structure	Search	Insert	Delete	Remarks
Unordered Array	$O(n)$	$O(1)$	$O(1)$	The time to Insert or Delete an item is less than the time after the location of the item has been ascertained by Search.
Ordered Array	$O(\log n)$	$O(n)$	$O(n)$	
Unordered List	$O(n)$	$O(1)$	$O(1)$	
Ordered List	$O(n)$	$O(1)$	$O(1)$	
Binary Search Tree	$O(h)$	$O(1)$	$O(1)$	

- Between an array and a list, there is a trade-off between search and insertion complexity



IIT Madras
Hyderabad

Search Data Structures (2)

Module 42

Partha Pratim Das

Objectives & Outline

Balanced BST

1.1.4. Time

Search

Insert

Delete

Example

Discussion

Module Summary

- Worst case time (n data items in the data structure):

Data Structure	Search	Insert	Delete	Remarks
Unordered Array	$O(n)$	$O(1)$	$O(1)$	The time to Insert or Delete an item is constant time after the location of the item has been ascertained by Search.
Ordered Array	$O(\log n)$	$O(n)$	$O(n)$	
Unordered List	$O(n)$	$O(1)$	$O(1)$	
Ordered List	$O(n)$	$O(1)$	$O(1)$	
Binary Search Tree	$O(h)$	$O(1)$	$O(1)$	

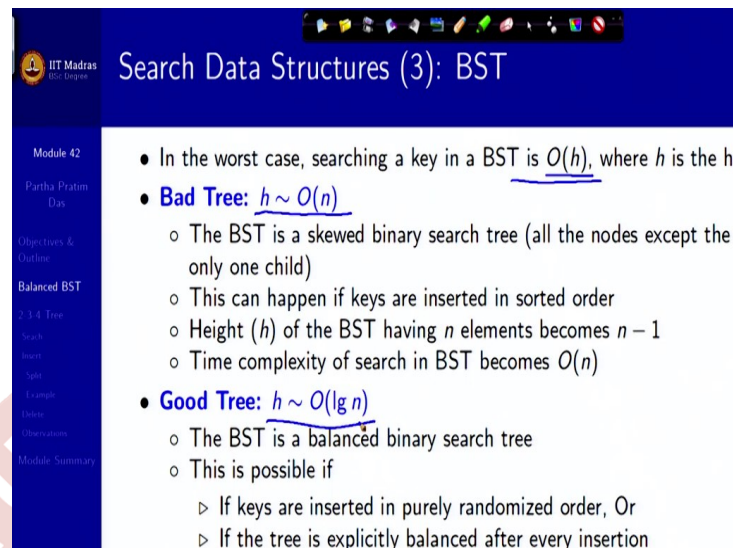
- Between an array and a list, there is a trade-off between search and insert/delete complexity

And this is what we have seen the trade-off between different of these and this is the key results that we have come to learn is binaries, in a binary search tree the search is order h , order height given the fact that the height is between these ranges and once you have been able to search then in additional order one time we can do an insert and delete.

Now, the point is, if you as you see the bounce on h , you have noted this earlier also that if you are not careful then a binary search tree, the height of a binary search tree could tend to become n , if it becomes n then since this is order h we lose all the advantages of doing a conceptual binary search, we will fall to order n .

So, we have to do something to make sure as we keep on doing different inserts and deletes, the height of the tree remains of the order of $\log n$ and if we can ensure that, guarantee that then we say that the binary tree is balanced.

(Refer Slide Time: 04:01)

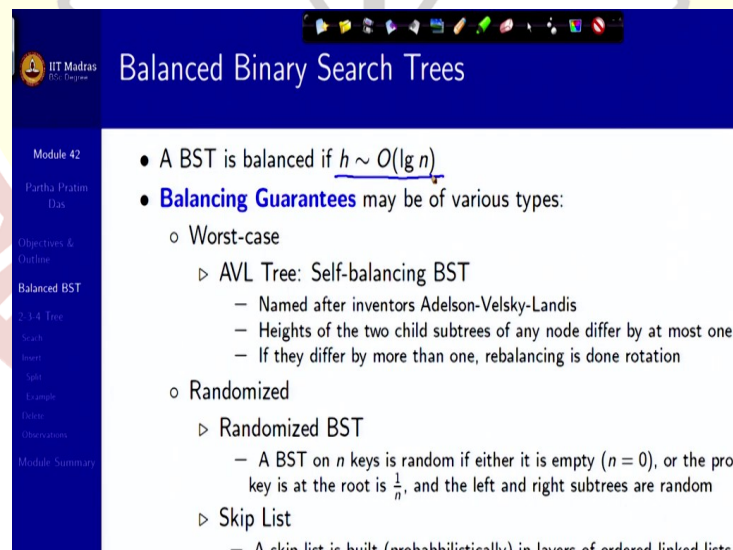


Search Data Structures (3): BST

- In the worst case, searching a key in a BST is $O(h)$, where h is the height of the tree.
- **Bad Tree:** $h \sim O(n)$
 - The BST is a skewed binary search tree (all the nodes except the root have only one child)
 - This can happen if keys are inserted in sorted order
 - Height (h) of the BST having n elements becomes $n - 1$
 - Time complexity of search in BST becomes $O(n)$
- **Good Tree:** $h \sim O(\lg n)$
 - The BST is a balanced binary search tree
 - This is possible if
 - ▷ If keys are inserted in purely randomized order, Or
 - ▷ If the tree is explicitly balanced after every insertion

So, again just to recap, this is your basic fundamental result that the search key in the worst case in a BST will be searched in order h time, trees may be bad where we have order n and trees may be good where we have order $\log n$, this way specifically this slide we discussed in the module we were discussing this I just replicated, so that the whole thing stays in your mind.

(Refer Slide Time: 04:36)



Balanced Binary Search Trees

- A BST is balanced if $h \sim O(\lg n)$.
- **Balancing Guarantees** may be of various types:
 - Worst-case
 - ▷ AVL Tree: Self-balancing BST
 - Named after inventors Adelson-Velsky-Landis
 - Heights of the two child subtrees of any node differ by at most one
 - If they differ by more than one, rebalancing is done rotation
 - Randomized
 - ▷ Randomized BST
 - A BST on n keys is random if either it is empty ($n = 0$), or the probability that a key is at the root is $\frac{1}{n}$, and the left and right subtrees are random
 - ▷ Skip List
 - A skip list is built (probabilistically) in layers of ordered linked lists


IIT Madras
 Mr. Dasgupta

Balanced Binary Search Trees

Module 42
 Partha Pratim Das
 Objectives & Outline
Balanced BST
 1.1-4. Tree
 Search
 Insert
 Split
 Example
 Delete
 Observations
 Module Summary

- A BST is balanced if $h \sim O(\lg n)$
- **Balancing Guarantees** may be of various types:
 - Worst-case ✓
 - ▷ AVL Tree: Self-balancing BST
 - Named after inventors Adelson-Velsky-Landis
 - Heights of the two child subtrees of any node differ by at most one
 - If they differ by more than one, rebalancing is done rotation
 - Randomized
 - ▷ Randomized BST
 - A BST on n keys is random if either it is empty ($n = 0$), or the probability that the root is at the root is $\frac{1}{n}$, and the left and right subtrees are random
 - ▷ Skip List
 - A skip list is built (probabilistically) in layers of ordered linked lists


IIT Madras
 Mr. Dasgupta

Balanced Binary Search Trees

Module 42
 Partha Pratim Das
 Objectives & Outline
Balanced BST
 1.1-4. Tree
 Search
 Insert
 Split
 Example
 Delete
 Observations
 Module Summary

- A BST is balanced if $h \sim O(\lg n)$
- **Balancing Guarantees** may be of various types:
 - Worst-case
 - ▷ AVL Tree: Self-balancing BST
 - Named after inventors Adelson-Velsky-Landis
 - Heights of the two child subtrees of any node differ by at most one
 - If they differ by more than one, rebalancing is done rotation
 - Randomized ✓
 - ▷ Randomized BST ✓
 - A BST on n keys is random if either it is empty ($n = 0$), or the probability that the root is at the root is $\frac{1}{n}$, and the left and right subtrees are random
 - ▷ Skip List
 - A skip list is built (probabilistically) in layers of ordered linked lists


IIT Madras
 Mr. Dasgupta

Balanced Binary Search Trees

Module 42
 Partha Pratim Das
 Objectives & Outline
Balanced BST
 1.1-4. Tree
 Search
 Insert
 Split
 Example
 Delete
 Observations
 Module Summary

- A BST is balanced if $h \sim O(\lg n)$
- **Balancing Guarantees** may be of various types:
 - Worst-case
 - ▷ AVL Tree: Self-balancing BST
 - Named after inventors Adelson-Velsky-Landis
 - Heights of the two child subtrees of any node differ by at most one
 - If they differ by more than one, rebalancing is done rotation
 - Randomized
 - ▷ Randomized BST
 - A BST on n keys is random if either it is empty ($n = 0$), or the probability that the root is at the root is $\frac{1}{n}$, and the left and right subtrees are random
 - ▷ Skip List
 - A skip list is built (probabilistically) in layers of ordered linked lists

IIT Madras
Balanced Binary Search Trees

Module 42
Partha Pratim Das
Objectives & Outline
Balanced BST
1.1.4. Tree
Search
Insert
Split
Example
Delete
Operations
Module Summary

- A BST is balanced if $h \sim O(\lg n)$
- **Balancing Guarantees** may be of various types:
 - Worst-case
 - ▷ AVL Tree: Self-balancing BST
 - Named after inventors Adelson-Velsky-Landis
 - Heights of the two child subtrees of any node differ by at most one
 - If they differ by more than one, rebalancing is done rotation
 - Randomized
 - ▷ Randomized BST
 - A BST on n keys is random if either it is empty ($n = 0$), or the probability that the root is $\frac{1}{n}$, and the left and right subtrees are random
 - ▷ Skip List
 - A skip list is built (probabilistically) in layers of ordered linked lists

So, the question is how to only work with good trees. Good trees that will remain balanced. Now in this, we kind of note though we will not go deeper into this that balancing is about giving a guarantee, you know you want this. Now the question is what are the ways you can guarantee that the height will be of the order of $\log n$ and there are different guarantees strategies that have been worked out that have happened.

One is called the worst case guarantee, which is what we have learned from that complexity of the algorithm that in the worst case, this must be guaranteed. So, which is the set of self-balancing BSTs primary being AVL tree, which is also the oldest of this data structures, which ensures that the height of two subtrees of a node, if you take any node, the height of the two sub trees that are rooted at this node that the children of the node, the you have a node, you have two children left and you have two sub trees, left subtree and right subtree having height h_L and h_R at every node, these heights of the left and right subtrees must not differ by more than 1.

If they differ by more than 1 because you have entered, you have inserted, they were the difference was 1 and you have inserted a key at the leaf node so that height has increased, then you have to do some rebalancing and there is a whole lot of strategy by which you can rotate a tree and balance it.

This is not an algorithm's course, so will not get into this. But there is a different variants of the self-balancing BST, which guarantee that you will always have the worst case, always have h be of the order of $\log n$, in the worst case, it will be good to have that but it has a lot of costs as well.

In actual terms, first of all doing these rotations are non-trivial and the actual cost of rotation in the physical runtime could be quite a lot. There is a lot of overhead and so on. So, comes a very different concept, which is from the domain of randomised algorithms. Some of these are slowly getting implemented in database context also, but I am currently talking only from the algorithms context.

So, which gives rise to say, okay, why did we get bad trees? Because we kept on possibly inserting in a particular path. Now, what, how do we get a good tree? If we without doing anything without doing AVL or anything, how can I get a good tree, I can get a good tree, because if my data is purely random, if my data is purely random, then when I am inserting, there is a equal probability of the data to go to the left child or to go to the right child.

Recursively at any level of the tree, if that happens always, then I am doing equal amount of insert on the left and right subtree of the entire tree at every node, which automatically guarantees that from leaf upwards, every sub tree is balanced and if every sub tree is balanced, the whole tree is balanced it is order $\log n$.

But the fact of the matter is that most often the data is biased it is not random. So, if we just follow the data in the order in which it comes, I cannot get a balanced binary search tree. So, that is where, it this concept of beautiful concept of randomised BST come which say that turn the table around, the data may be biased, but what you can do is you can, randomly decide which side you will enter it based on the value of the data and certain factors.

So, that is I mean I will not go into the depth, if you study the book, corresponding book you will find that this guarantees that the probability of, so you do not talk about a worst-case guarantee that h is always order $\log n$, but what you say is, the probability of h not being more than order $\log n$ is extremely small.

So, that is another approach. Another randomised strategy is skip list, I mentioned this where you keep multiple levels of ordered link list much in the way you are doing multi-level indexing, in the bottom most list, you keep all entries, in the second you keep maybe alternate, half the entries in the third level, you keep one fourth of the entries and so on so forth.

And how you choose which entries to keep at what level you use the randomization for that which guarantees actually, if you use a true randomization it guarantees that it will have an effective search time, which is order $\log n$, very strictly speaking, I should not be calling this

as a binary search tree because it has a different kind of structure. But it is a linked search structure which follows the binary searching algorithm. So, I prefer to call it that way.

And finally, you have splay, which look at the entire do not look at any specific size, height of the tree or anything, it just says that, if I do 10,000, inserts and deletes, then whatever is the cost that I get, if I divide this by 10,000, if that average is order $\log n$, then I am happy. So, that is amortization. So, these are different ways to keep things balanced.

(Refer Slide Time: 10:33)

Balanced Binary Search Trees (2)

- These data structures have optimal complexity for the required operations
 - Search: $O(\lg n)$
 - Insert: Search + $O(1)$: $O(\lg n)$
 - Delete: Search + $O(1)$: $O(\lg n)$
- And they are:
 - Good for in-memory operations ✓
 - Work well for small volume of data ✓
 - Has complex rotation and / or similar operations ✓
 - Do not scale for external data structures ✓

Now, the fact of the matter is that these all-guarantee order $\log n$. But do they satisfy our requirement now, they are good for in memory operations. They work well with small volume of data, small compared to what a database need, they will work well with maybe 10,000; 100,000; 200,000 data, but not with 10 million data, because you cannot fit them in the memory.

They have complex rotation or similar operation like randomization, random rotation, all of these, which makes it very difficult to scale them up and consequently, they do not scale, they do not directly map to external data structures. So, this is the state of the art in terms of algorithms in memory data structure, but we does does not give you as a solution for our database file structure of the database physical structure.

(Refer Slide Time: 11:44)

2-3-4 Tree

Module 42
Partly Praton
Dias
Objectives &
Outline
Balanced BST
2-3-4 Tree
Stack
Queue
Example
Complexity
Observations
Module Summary

2-3-4 Tree

- All leaves are at the same depth (the bottom level).
 - Height, h , of all leaf nodes are same
 - ▷ $h \sim O(\lg n)$
 - ▷ Complexity of search, insert and delete: $O(h) \sim O(\lg n)$
- All data is kept in sorted order
- Every node (leaf or internal) is a 2-node, 3-node or a 4-node (based on the number of links or children), and holds one, two, or three data elements, respectively
- Generalizes easily to larger nodes ✓
- Extends to external data structures

Handwritten notes: $n \approx 2^{h-1}$, $n \approx 2^{h-1}$

So, we look at another data structure, which is also in memory called 2-3-4 tree, which can give us a very good solution. What is the concept? The concept is we are trying to do something very different, in a different approach, what we say is, we hear, we guarantee or we build a tree in a way that all leaves are at the same level depth, all leaves are at the same depth.

So, if all lives at the same depth, then naturally the height of the tree has to be $\log n$. That is, but we had observed that all the leaves can be at the leaves provided is n is of the order of 2 to the power h minus 1 and then it will again happen at 2 to the power h plus 1 minus 1. So, what if n is greater than this and less than this within this range?

Naturally, you cannot guarantee that all of them will be at the leaf level, because you will not have enough space to keep them. So, that is the basic challenge. Now, you want to keep the data in sorted order and that is how you will enter it. So, in the leaf level, if you read it from left to right is a sorted order. That is not difficult.

The basic question is where does, if n nodes, n data there, then where does these extra data go in without increasing the height of the tree, very simple, you can let nodes accommodate more than one data values as long as they satisfy certain properties. So, instead of 1 node type which we have so far, but the node has only one data value, we create three types of nodes.

A 2 node which has only one data value and two children, 3 node having two data values and three children, 4 node having three data values and four children. And the idea is whole thing can generalise to the larger nodes very easily and therefore can extend to the external data structure.

(Refer Slide Time: 14:11)

2-3-4 Trees

- Uses 3 kinds of nodes satisfying key relationships as shown below:
 - A 2-node must contain a single data item (S) and two links
 - A 3-node must contain two data items (S, L) and three links
 - A 4-node must contain three data items (S, M, L) and four links
 - A leaf may contain either one, two, or three data items

Diagram illustrating node types and search key relationships:

- 2-node:** Contains a single data item (S). Search keys are $< S$ and $> S$.
- 3-node:** Contains two data items (S, L). Search keys are $< S$, $S \text{ and } L$, and $> L$.
- 4-node:** Contains three data items (S, M, L). Search keys are $< S$, $S \text{ and } M$, $M \text{ and } L$, and $> L$.

Footer: सिद्धिर्भवति कर्मजा

Module 42

Partha Pratim Das

Objectives & Outline

Balanced BST

2-3-4 Tree

Search

Insert

Split

Example

Notes

Observations

Module Summary

2-3-4 Trees

- Uses 3 kinds of nodes satisfying key relationships as shown below:
 - A 2-node must contain a single data item (S) and two links
 - A 3-node must contain two data items (S, L) and three links
 - A 4-node must contain three data items (S, M, L) and four links
 - A leaf may contain either one, two, or three data items

Module 42

Partha Pratim Das

Objectives & Outline

Balanced BST

2-3-4 Tree

Search

Insert

Split

Example

Notes

Observations

Module Summary

2-3-4 Trees: Search

- Search
 - Simple and natural extension of search in BST

Module 42

Partha Pratim Das

Objectives & Outline

Balanced BST

2-3-4 Tree

Search

Insert

Split

Example

Notes

Observations

Module Summary

2-3-4 Trees

- Uses 3 kinds of nodes satisfying key relationships as shown below:
 - A 2-node must contain a single data item (S) and two links
 - A 3-node must contain two data items (S, L) and three links
 - A 4-node must contain three data items (S, M, L) and four links
 - A leaf may contain either one, two, or three data items

So, let us look at the concrete proposal S is a 2-node which has one single data item and 2 search keys like the BST, a 2 node has two S and the single and the large and it has 3. So, this is, these are the data items which are less than S, these are data items which are more than L and these are data items between S and L you have to satisfy this.

So, this is 2. This is a 3-node having 2 data items or you can have a 4-node having 3 data items. S, M and L, so that on the, there are four links, the first one is less than L, S the second one is between S and M here, the third one is between M and L and the fourth one is L greater than L. So, this is how you can create this, you want to have nodes and link them.

Now, the rest of the structure is if you do this, then searching on this will be simple, because you follow the same strategy, you are at a single value you were checking, at a single value earlier you were checking whether it is less or more and going, here we will have to check that whether it is less than this, between this or more than this and go accordingly. So, the same thing will happen for the 4-node.

(Refer Slide Time: 15:49)

2-3-4 Trees: Insert

- Insert
 - Search to find expected location
 - ▷ If it is a 2 node, change to 3 node and insert
 - ▷ If it is a 3 node, change to 4 node and insert
 - ▷ If it is a 4 node, split the node by moving the middle item to a new node and insert
 - Node Splitting
 - ▷ A 4-node is split as soon as it is encountered during a search for a leaf
 - ▷ The 4-node that is split will
 - Be the root, or a child of the root

So, search is a natural extension. Now, the question is how do you insert, now to insert you do something very simple, you try to find out by search where your value to insert was expected, like the binary search tree that is easy to do. Now, what are the possibilities, there are three possibilities, one that could be a 2-node having one value.

If it is a 2 node, you do not do anything, you do not do any going for the linking anything, you just change the 2-node to a 3-node, if it is a 3-node, then you change it to a 4-node, two values were there and your new value inserted. So, no height change or anything is required.

The question is if it is a 4-node, then you cannot put more values because you have assumed that there can be only up to 3, 3 keys in a node maximum. So, what you do of these 3 values S, M and L, you take this out and insert it to its parent, go up insert it in the parent, when that happens, then you have 2 nodes, 2 values you put the new value.

Subsequent question is, so this is called kind of splitting the node. Now, subsequent question is what happens in the parent, because you want to insert it there. Now, if there is no parent, you are doing at the root, then you make M the new root, you may have a 2 parent, otherwise you may have a 2 parent or you may have 3 parent, because if you, let us assume that we are not letting a 4-node stay by itself, when we encounter it.

So, up onwards there is no 4-node, if there was a 4-node, we have already taken care of it, taken its middle value put to the parent and recursively up to the root. So, it can either be the root itself, it can be a 2-parent node or a 3-parent node.

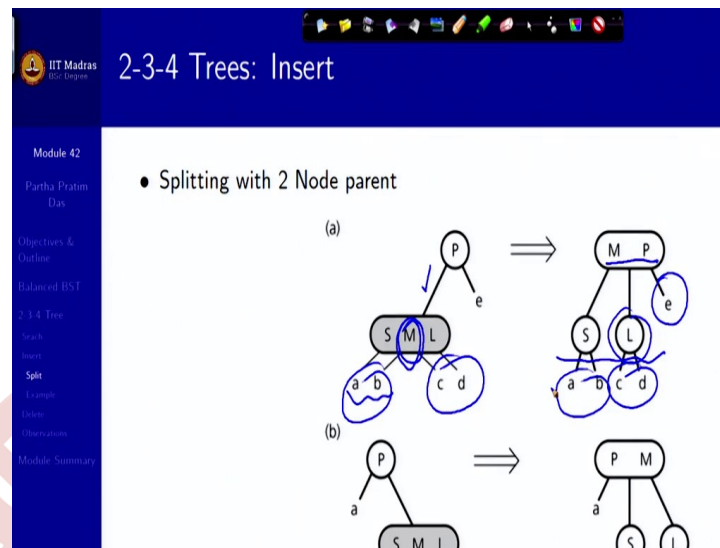
(Refer Slide Time: 18:09)



So, what will happen if it is already the 4-node is already the root, very simple, you split make M the root, S the left child here, L the right child here, you have a b c d, a b becomes links of S, c d becomes links of L, you please note that you are maintaining the order, because now if anything is between S and M you will go on this side, if anything is between M and L you will go this side.

And so, therefore if you have to come to c, what you have is, it is more than M and less than L, this is exactly what c was and so on. So, convince yourself with this split, but with this a 4-node is replaced by three 2 nodes.

(Refer Slide Time: 19:06)



If it has a two parent, when you, so you have to split this, put this M put it to the parent. Now, there are two possibilities. It can be less than the parent or it can be more than the parent. If it is, I am sorry. There are, this is, in this case the 4-node is a left child. So, it is less than the parent.

Therefore, when I convert this 2-node into a 3-node, it will occur to the left and then I, all that I have to do is make S and L the two other nodes of this 3-node, e will remain as it is because it is greater than P, there we are not involved. Something between M and P would have been between c d. So, L will point to c d and S will point to a b, because it is less than S or between S and M. So, this converts the 3-node, the 1 node at the parent into a 2-node and splits the 4-node exactly the similar logic will split it, if it is on the right child.

(Refer Slide Time: 20:40)

IIT Madras
For All

Module 42

Partha Pratim Das

Objectives & Outline

Balanced BST

2-3-4 Tree

Search

Insert

Split

Example

Exercise

Observations

Module Summary

2-3-4 Trees: Insert

- Splitting with 3 Node parent

(a)

(b)

IIT Madras
For All

Module 42

Partha Pratim Das

Objectives & Outline

Balanced BST

2-3-4 Tree

Search

Insert

Split

Example

Exercise

Observations

Module Summary

IIT Madras
For All

Module 42

Partha Pratim Das

Objectives & Outline

Balanced BST

2-3-4 Tree

Search

Insert

Split

Example

Exercise

Observations

Module Summary

2-3-4 Trees: Insert

- Insert
 - Search to find expected location
 - If it is a 2 node, change to 3 node and insert
 - If it is a 3 node, change to 4 node and insert
 - If it is a 4 node, split the node by moving the middle item to parent and insert
 - Node Splitting
 - A 4-node is split as soon as it is encountered during a search for a leaf
 - The 4-node that is split will
 - Be the root, or

IIT Madras
For All

Module 42

Partha Pratim Das

Objectives & Outline

Balanced BST

2-3-4 Tree

Search

Insert

Split

Example

Exercise

Observations

Module Summary

And finally, if you have a 3 node as a parent, you just extend that logic in the left you take care of it like this, if it is on the middle node, you take care of by putting it in M and if it is on the right you take it here. So, in each every, each one of these cases, the 3 node becomes a 4 node, you may choose to propagate to split this or you may choose to keep it till you come across this 4 node later on.

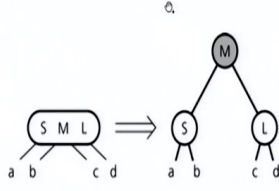
So, with this what we have guaranteed that in none of these if you, if I now go back, if I now go back, if I now go back two steps, then with this insert a 2 node change to 3 node height did not change, 3 node change to 4 node, height did not change, 4 node change through, change through splitting.

(Refer Slide Time: 21:39)

IIT Madras
Module 42
Partha Pratim Das
Objectives & Outline
Balanced BST
2-3-4 Tree
Search
Insert
Split
Example
Delete
Observations
Module Summary

2-3-4 Trees: Insert

- Splitting at Root



And in that splitting, if it becomes a root then the height changes. But, the height changes for all the leaf nodes, because you are changing the height by 1 at the root. So, all paths to all leaf nodes increased by 1. So, h has become $h+1$ and it was already everything is filled at least one node is there in this tree of height h .

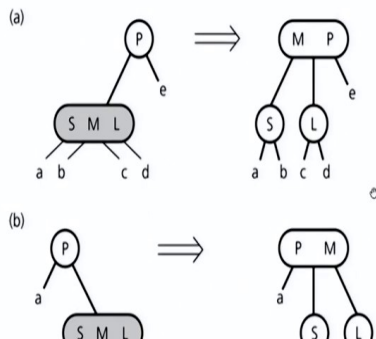
So, you have 2 to the power h minus 1 or more nodes. Therefore, this height cannot be more than $\log n$. This is the only case where your height changes, not like in ordinary BST where you change, you may change the height almost every time you insert a node, that is the beauty of the 2-3-4 tree.

(Refer Slide Time: 22:27)

IIT Madras
Module 42
Partha Pratim Das
Objectives & Outline
Balanced BST
2-3-4 Tree
Search
Insert
Split
Example
Delete
Observations
Module Summary

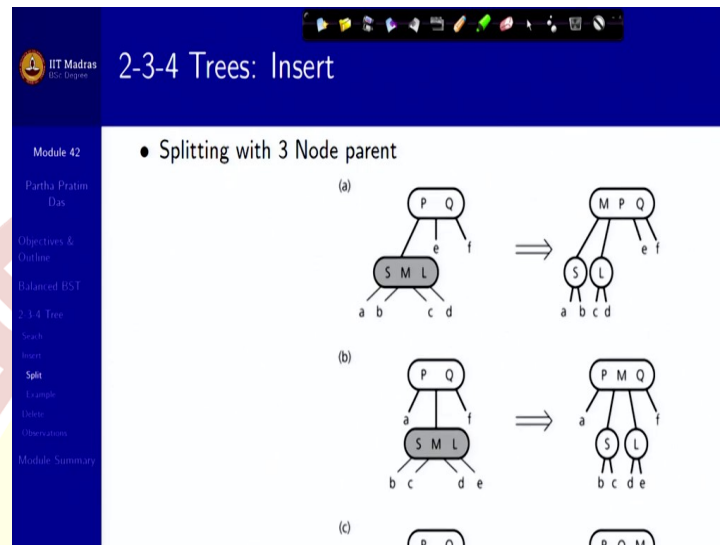
2-3-4 Trees: Insert

- Splitting with 2 Node parent



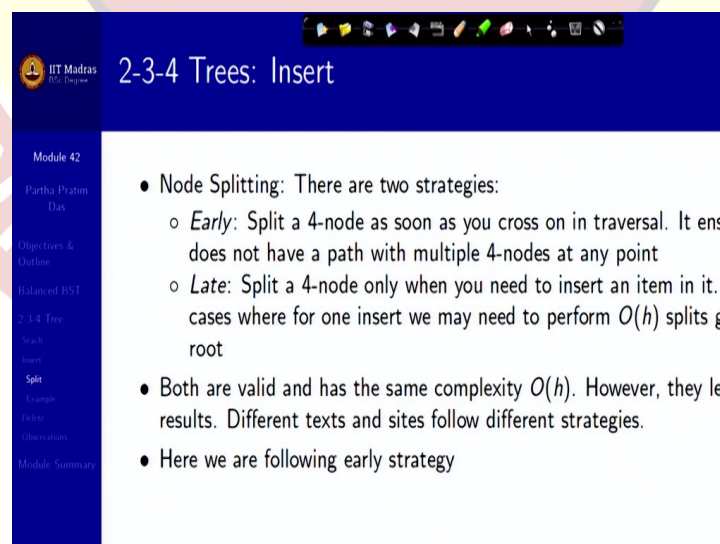
If you look into the other cases of split an insert in the parent the height is not changing, you are just adjusting points, you are keeping more space to accommodate more and more nodes, but height is not changing in any of this case.

(Refer Slide Time: 22:42)



So, there is only one place where the height changes in a 2-3-4 tree where you split a node which is a 4 node at the root and then the height increases. So, by this we are come to the conclusion that what we assume that all nodes can be at the same level is possible.

(Refer Slide Time: 23:02)



So, you could do it, do this node splitting by two strategies, you can do it early, that is split a node as soon as you come across it on the traversal. So, that when you are going back through an insert, you do not expect any other 4 node to come across or it could be a late strategy that

you split a 4 node, only when you need to insert an item, there is no trade off, I mean there is no specific trade-off between these two, because both finally, will add a height when you split the 4 node at the root, both are valid have the same complexity.

However, these different strategies will produce different results there could be different trees, but the same performance. For our illustration here we are following the early strategy.

(Refer Slide Time: 23:54)

The image shows two screenshots of a presentation slide titled "2-3-4 Trees: Insert: Example". The slide is from IIT Madras, Module 42, by Partha Pratim Das. The slide content is as follows:

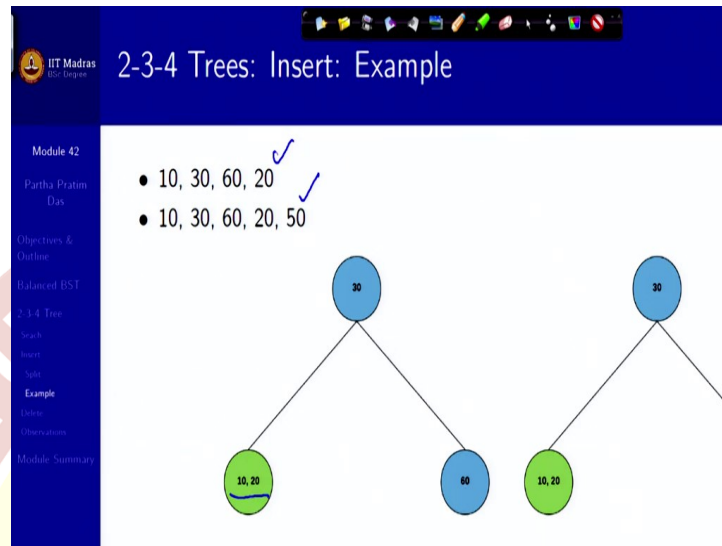
- Insert 10, 30, 60, 20, 50, 40, 70, 80, 15, 90, 100
- 10
- 10, 30
- 10, 30, 60
- Split for 20

The first screenshot shows a tree structure with a root node 30 and three child nodes: 10, 10, 30, and 10, 30, 60. The second screenshot shows the same tree structure, but with arrows indicating the insertion of 20 into the 10, 30, 60 node, and a new parent node 30 being created above it.

So, quickly an example is in order. So, this is the order in which I want to insert the keys in and make a 2-3-4 tree. So, I insert 10 then I get a 1 node at the 2 node at the root, insert 30 this becomes a 3 node, insert 60 this becomes a 4 node, now following an early strategy. So, when we want to, this has been so, want to do, insert 20 as soon as we come across this we have to split and what we split, get in the split is the 30 goes up to the parent which becomes

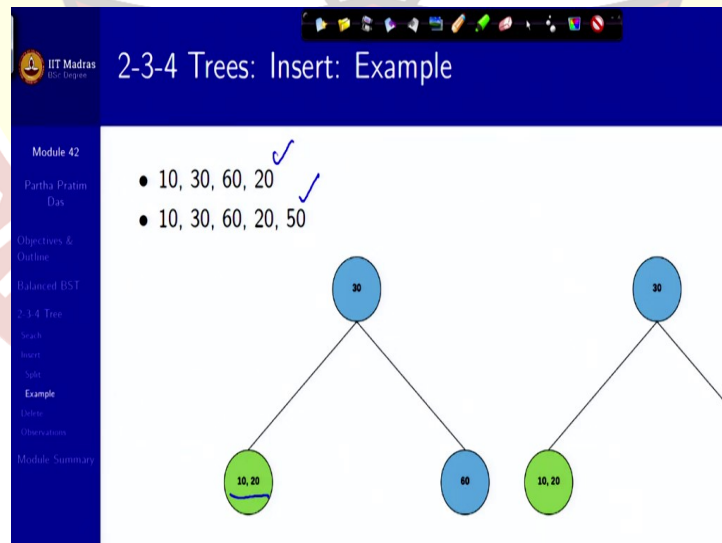
a new route. So, 30, 10, 60 is the new tree, where I can now insert 20 which I was not able to do in this tree.

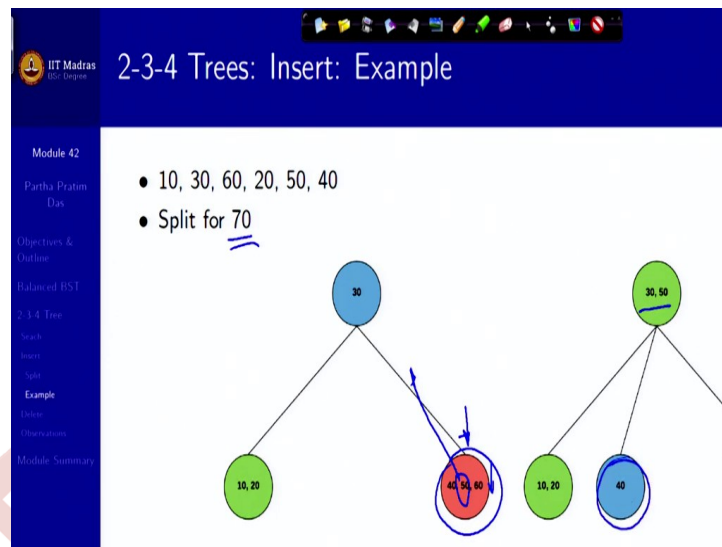
(Refer Slide Time: 24:54)



So, the next step is inserting 20, so I have inserted 20, it is less than 30 and comes on the left child, next is to insert 50. So, 50 is greater than this and less than 60. So, it comes on the left, no adjustments needed.

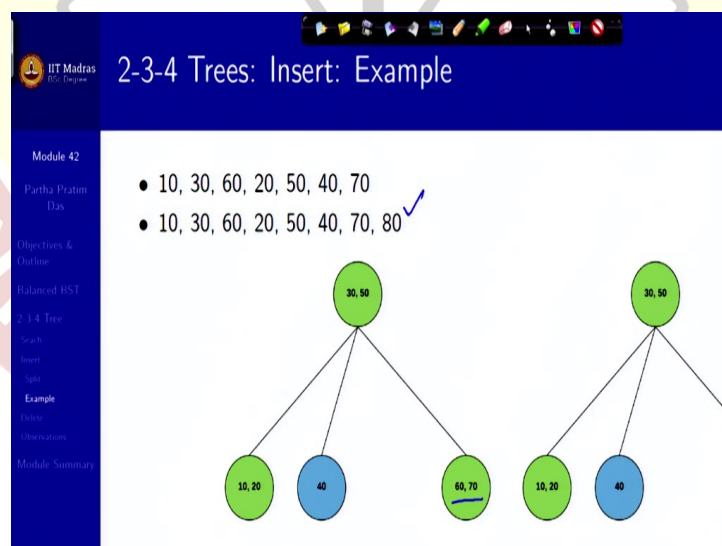
(Refer Slide Time: 25:13)





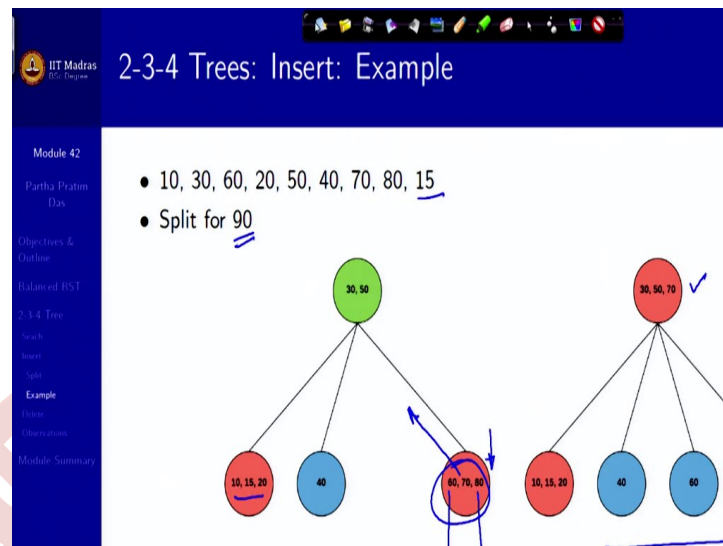
I go on, I am inserting 40 as I insert 40, as I insert 40, 40 is greater than 50, 30 less than 50. So, it comes here and I have 40 inserted at this point. Now, my next target is to insert 70, if I want to insert 70, then where will it come, greater than 30, it will come in this 4 node, it is greater than 60, so this is where I have to insert. So, which means I have to split this. So, I split, if I split middle node, middle value goes up, so this 2 node becomes a 3 node and this 40 comes here separately, 60 comes here separately.

(Refer Slide Time: 26:08)



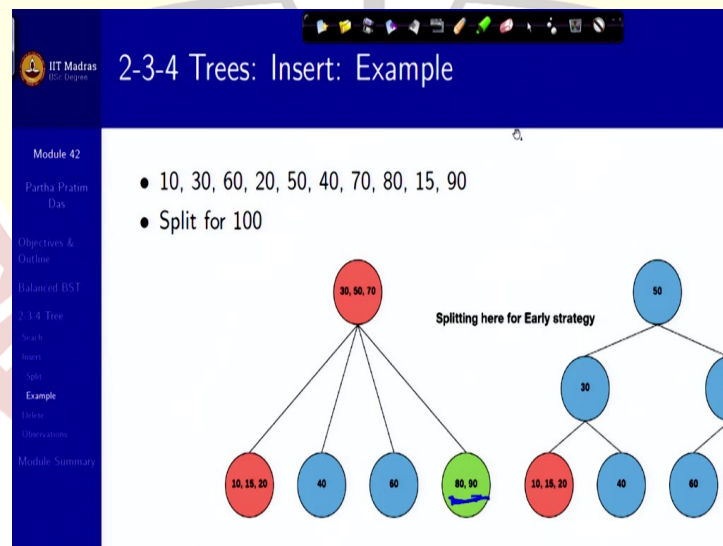
Now, we do not have a 4 node I can easily insert 70 which will come here, then I have 80, I insert 80 it comes here, mind you with all these, at the very beginning after the three insertions, the height has change, in all these the height is not changing, it remains the same.

(Refer Slide Time: 26:29)



Now, when I was able to insert 80 and then I have to insert 15 which goes in here not a problem, I have to insert 90 which has to go here. So, which means this has to split. If this has to split 70 has to go up. So, it goes up, this becomes a 4 node. I have 40, 60, 80 coming as 2 nodes. So, now I am ready to insert 90.

(Refer Slide Time: 27:04)

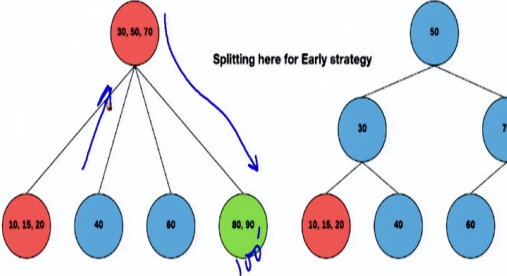



IIT Madras
For Engineers

2-3-4 Trees: Insert: Example

Module 42
Partha Pratim Das
Objectives & Outline
Balanced BST
2-3-4 Tree
Search
Insert
Split
Example
Delete
Observations
Module Summary

- 10, 30, 60, 20, 50, 40, 70, 80, 15, 90
- Split for 100

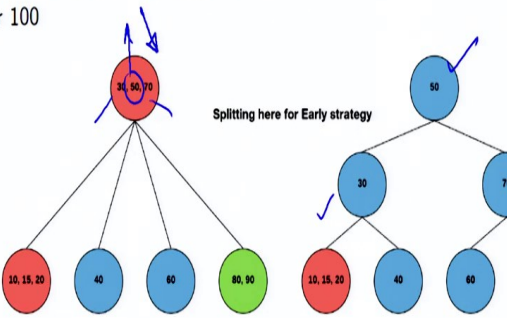



IIT Madras
For Engineers

2-3-4 Trees: Insert: Example

Module 42
Partha Pratim Das
Objectives & Outline
Balanced BST
2-3-4 Tree
Search
Insert
Split
Example
Delete
Observations
Module Summary

- 10, 30, 60, 20, 50, 40, 70, 80, 15, 90
- Split for 100

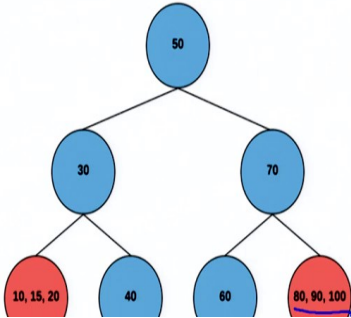



IIT Madras
For Engineers

2-3-4 Trees: Insert: Example

Module 42
Partha Pratim Das
Objectives & Outline
Balanced BST
2-3-4 Tree
Search
Insert
Split
Example
Delete
Observations
Module Summary

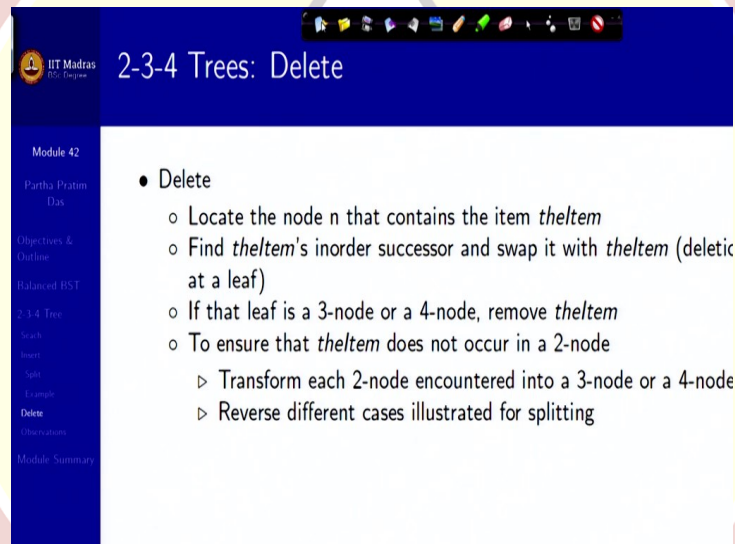
- 10, 30, 60, 20, 50, 40, 70, 80, 15, 90, 100



So, I have inserted 90 and then I have to insert 100. So, where will 100 go? If I do 100 then it will come here. So, this is, now the question is you may ask us to here, the basic point is that I could have on this tree come here and placed 100 here, it is possible. But that would mean that I am following a late strategy. So, I will do the splitting next when something pops up from below to cause the root to split, but I am calling an early strategy.

So, what does that mean? That means that when I am searching for inserting 100, the moment I come across a 4 node I split. So, I split this as I split this, this becomes the root, these are the two children, these are the two children. So again, from one, the height of this tree has now become 2 and then I insert 100. So, my 100 gets inserted here, but on that path, there is no 4 node. So, this is the basic strategy for 2-3-4 tree insertion.

(Refer Slide Time: 28:48)

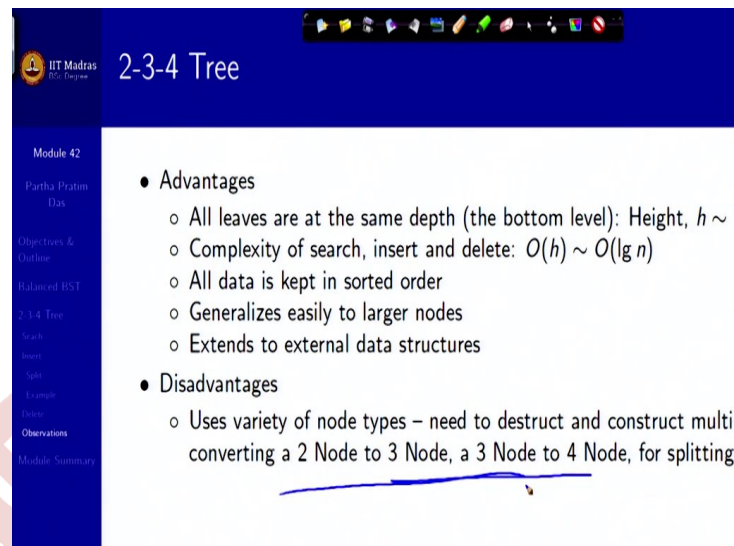


The screenshot shows a presentation slide from IIT Madras. The title is "2-3-4 Trees: Delete". The slide content is as follows:

- Delete
 - Locate the node n that contains the item $theltem$
 - Find $theltem$'s inorder successor and swap it with $theltem$ (delete at a leaf)
 - If that leaf is a 3-node or a 4-node, remove $theltem$
 - To ensure that $theltem$ does not occur in a 2-node
 - ▷ Transform each 2-node encountered into a 3-node or a 4-node
 - ▷ Reverse different cases illustrated for splitting

You can work out the 2-3-4 tree deletion in a similar way, all that you need to do is to whenever here, whenever you got a 4 node and you wanted to insert, you had to remove, split and insert in the parent, you will now have to do the reverse, you will have to merge the nodes.

(Refer Slide Time: 29:10)



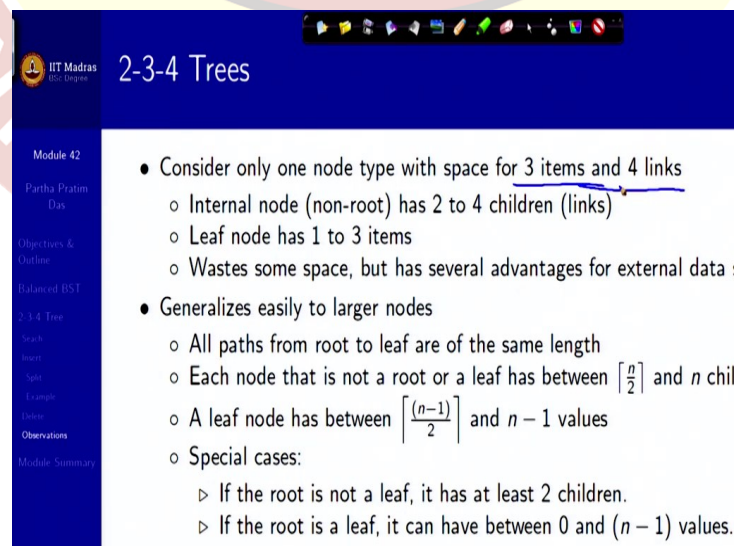
2-3-4 Tree

- Advantages
 - All leaves are at the same depth (the bottom level): Height, $h \sim$
 - Complexity of search, insert and delete: $O(h) \sim O(\lg n)$
 - All data is kept in sorted order
 - Generalizes easily to larger nodes
 - Extends to external data structures
- Disadvantages
 - Uses variety of node types – need to destruct and construct multiple node types converting a 2 Node to 3 Node, a 3 Node to 4 Node, for splitting

Now, so the advantages that we have is all leaves are at the same level complexity of all search is order h , which is $\log n$, data is kept in sorted order, it can be generalised to larger nodes which is, because 2-3-4 was just the minimum that you can do. But I can do nodes of a number of types.

But certainly, it has the disadvantage that I have multiple types of nodes. So, that causes because I have to always change from a 2 node to a 3 node, 3 node to a 4 node, 4 node to multiple 2 node. So, I have to keep on actually destructing and constructing nodes which becomes an overhead.

(Refer Slide Time: 29:53)



2-3-4 Trees

- Consider only one node type with space for 3 items and 4 links
 - Internal node (non-root) has 2 to 4 children (links)
 - Leaf node has 1 to 3 items
 - Wastes some space, but has several advantages for external data
- Generalizes easily to larger nodes
 - All paths from root to leaf are of the same length
 - Each node that is not a root or a leaf has between $\lceil \frac{n}{2} \rceil$ and n children
 - A leaf node has between $\lceil \frac{(n-1)}{2} \rceil$ and $n-1$ values
 - Special cases:
 - ▷ If the root is not a leaf, it has at least 2 children.
 - ▷ If the root is a leaf, it can have between 0 and $(n-1)$ values.


IIT Madras
 For Engineers

2-3-4 Trees

Module 42
 Partha Pratim Das
 Objectives & Outline
 Balanced BST
 2-3-4 Tree
 Search
 Insert
 Split
 Example
 Notes
 Observations
 Module Summary

- Consider only one node type with space for 3 items and 4 links
 - Internal node (non-root) has 2 to 4 children (links)
 - Leaf node has 1 to 3 items
 - Wastes some space, but has several advantages for external data
- Generalizes easily to larger nodes
 - All paths from root to leaf are of the same length ✓
 - Each node that is not a root or a leaf has between $\lceil \frac{n}{2} \rceil$ and n children
 - A leaf node has between $\lceil \frac{n-1}{2} \rceil$ and $n-1$ values
 - Special cases:
 - If the root is not a leaf, it has at least 2 children.
 - If the root is a leaf, it can have between 0 and $(n-1)$ values.


IIT Madras
 For Engineers

2-3-4 Trees

Module 42
 Partha Pratim Das
 Objectives & Outline
 Balanced BST
 2-3-4 Tree
 Search
 Insert
 Split
 Example
 Notes
 Observations
 Module Summary

- Consider only one node type with space for 3 items and 4 links
 - Internal node (non-root) has 2 to 4 children (links)
 - Leaf node has 1 to 3 items
 - Wastes some space, but has several advantages for external data
- Generalizes easily to larger nodes
 - All paths from root to leaf are of the same length ✓
 - Each node that is not a root or a leaf has between $\lceil \frac{n}{2} \rceil$ and n children ✓
 - A leaf node has between $\lceil \frac{n-1}{2} \rceil$ and $n-1$ values
 - Special cases:
 - If the root is not a leaf, it has at least 2 children.
 - If the root is a leaf, it can have between 0 and $(n-1)$ values.

So, the strategy could be that we just have 4 nodes. We have 3 items and 4 links and I use it as a 2 node or as a 3 node and I let some space go waste, it is not that kind of a premium, that is the crucial observation here. So, if I have that, if I have that wastage is there, so leaf node can have 1, 2, 3 items, whatever.

Now, this will generalise easily to larger nodes, again by the same strategy all paths from root to leaf are of the same length, each node that is not a root or a leaf, because at the root we cannot guarantee this, but what do you want to guarantee is I have at least half of the links filled up.

Otherwise, my wastage will be very, very high and the leaf node obviously can have number of values which are in this bound, the special case has to be for the leaf because the tree may not have so many nodes. So, this is the basic idea that you want to guarantee that all paths

from root to leaf are of same length. You want to keep multiple values in the same node making sure that you have every node at least half filled.

So, in case of a 2-3-4 tree n being 4 here, will make sure that other than the root, you must have at least 2 values or 3 values, not, I mean at least 4 children or 3 children, 2 values or 3 values. So, that will make sure that at least more than half the space is always utilised.

(Refer Slide Time: 32:27)

Module Summary

- Recapitulated the notions of Balanced Binary Search Trees as options in-memory search data structures
- Understood the issues relating to external data structures for persistence
- Explored 2-3-4 Tree in depth as a precursor to B/B+-Tree for an efficient structure for database and index tables

Slides used in this presentation are borrowed from <http://db-book> permission of the authors.

So, that is, that was a foundational idea I wanted to give you how you transition from a in memory data structure for search to a external data structure for search I will develop, we will develop on this idea more in the next module when we talk about B+ tree and B tree. Thank you very much for your attention and we will meet in the next module.